

Strings

08 July 2026 20:26

String is a sequence which is made up of one or more Unicode characters. Here the character can be a letter, digit, white space or any other symbol.

A string can be created by enclosing one or more characters in a single, double, or triple quote.

```
print('\u20b9')
```

₹

```
print('\u0048')
```

H

```
print('\u0040')
```

@

```
print('\u0041')
```

A

```
str1 = 'Hello! python'
```

```
str2 = "Shiv Ray"
```

```
Str3 = """Hello shiv
```

```
How r u?
```

```
Are you coding!
```

```
"""
```

```
Str3
```

```
'Hello shiv\nHow r u?\nAre you coding!\n'
```

```
print(Str3)
```

```
Hello shiv
```

```
How r u?
```

```
Are you coding!
```

Unicode values, they are 4 digit hexadecimal values.

← Triple quote is used for multiline string.

characters inserted by Python when we use triple quote.

```
str1
```

```
'Hello! python' ✓
```

```
str2
```

```
'Shiv Ray' ✓
```

```
Str3
```

Internally Python holds the string into single quotes.

```
str4 = '''I am coding
```

```
in Python
```

```
not in Java'''
```

```
str4
```

```
'I am coding\nin Python\nnot in Java'
```

```
print(str4)
```

```
I am coding
```

```
in Python
```

```
not in Java
```

0	1	2	3	4	5	6	7	8	9	10	11	12
H	e	l	l	o	!		p	y	t	h	o	n
-13	-12	-11	-10	-9	-8	-7	-6	-5	-4	-3	-2	-1

Accessing characters in a string

Each individual character in a string can be accessed using a technique called indexing. The index specifies the character to be accessed in the string and is written in square brackets([]). The index of the first character (from left) in the string is zero and the last character is N minus 1. Where N is the length of the string. If we give

index value out of this range then we get an index error. The index must be an integer (positive, 0 or negative).

'0' index represents the first character of the string and '-1' index represents the last character of the string.

For example:

```
str1
'Hello! python'
str1[0]
'H'
str1[1]
'e'
str1[2]
'l'
str1[-1]
'n'
```

```
len(str1)
13
```

```
str1[20]
```

Traceback (most recent call last):

File "<pyshell#26>", line 1, in <module>

```
str1[20]
```

IndexError: string index out of range

```
str1[-20]
```

Traceback (most recent call last):

File "<pyshell#27>", line 1, in <module>

```
str1[-20]
```

IndexError: string index out of range

Python allows an index value to be negative also. Negative indices are used when we want to access the character of the string from right to left. Starting from right hand side, the first character has the index as -1 and the last character has the index -N. Where N is the length of the string.

0	1	2	3	4	5	6	7	8	9	10	11	12
H	e	l	l	o	!		p	y	t	h	o	n
-13	-12	-11	-10	-9	-8	-7	-6	-5	-4	-3	-2	-1

Accessing the first character and the last character in another way:

```
n = len(str1)
n
13
str1[n-1]
'n'
str1[-n]
'H'
str1
'Hello! python'
```

String is Immutable

A string is an immutable data type. It means that the content of the string cannot be changed after it has been created. An attempt to do this would lead to an error.

```
str1
'Hello! python'
```

```
str1[0] = 'h'
Traceback (most recent call last):
  File "<pyshell#35>", line 1, in <module>
    str1[0] = 'h'
TypeError: 'str' object does not support item assignment
```

String Operations

As we know that String is a sequence of characters. Python allows certain operations on string data types, such as concatenation, repetition, membership, and slicing.

Concatenation: To concatenate means to join. Python allows us to join two strings using concatenation operator which is denoted by symbol +.

```
Example:
fname = 'Shiv'
lname = 'Ray'
fullName = fname + ' ' + lname
fname
'Shiv'
lname
'Ray'
fullName
'Shiv Ray'
```

Repetition

Python allows us to repeat the given string using repetition operator which is denoted by symbol *.

For example:

```
fname
'Shiv'
lname
'Ray'
fullName
'Shiv Ray'
char = '%'
newchars = char * 10
newchars
'%%%%%%%%%'
lname * 5
'RayRayRayRayRay'
6 * fname
'ShivShivShivShivShivShiv'
char
 '%'
fname
'Shiv'
```

For repetition:

one operand must be of type 'str'
and other operand must be of type 'int'.

Membership: Python has two membership operators, 'in' and 'not in'. The 'in' operator takes two strings and returns True if the first string appears as a substring in the 2nd string, otherwise it returns False.

For example:

```
str1
```

appears as a substring in the 2nd string, otherwise it returns False.

For example:

```
str1
'Hello! python'
'o' in str1
True
'y' in str1
True
'P' in str1
False
'Py' in str1
False
'py' in str1
True
```

The '**not in**' operator also takes two strings and returns True if the 1st string does not appear as the substring in the 2nd string.

Otherwise returns False.

For example:

```
str1
'Hello! python'
'x' not in str1
True
'p' not in str1
False
'py' not in str1
False
'pyc' not in str1
True
```

Slicing

In Python to access some part of string or substring, we use a method called slicing. This can be done by specifying an index range. Given a string STR one. The slice operation STR1 [N:M] Returns the part of the string STR1 starting from index N (Inclusive) And ending at M (exclusive). In other words, we can say that STR [N: M] Returns all the characters starting from STR1 [N] till STR [M-1]. The number of characters in a substring will always be equal to the difference of two indices M and N i.e., (M - N)

```
str1 = 'Computer Application'
```

For example:

```
str1 = 'Computer Application'
substr1 = str1[9:20]
substr1
'Application'
substr2 = str1[0:9]
substr2
'Computer'
```

0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18	19
C	o	m	p	u	t	e	r		A	p	p	l	i	c	a	t	i	o	n

```
substr2 = str1[0:9]
```

```
substr2
'Computer'
0 1 2 3 4 5 6 7 8 9 10 11 12 13 14 15 16 17 18 19
c o m p u t e r   A p p l i c a t i o n
-20 -19 -18 -17 -16 -15 -14 -13 -12 -11 -10 -9 -8 -7 -6 -5 -4 -3 -2 -1
```

```
Str_var[start:end:skip]
```

```
substr3
'Application'
substr3[len(substr3)-1:-1]
''
substr3[len(substr3)-1:0]
''
```

optional optional optional
This is required when 3rd option is given.
is compulsory

Reversing the string:

```
substr3[len(substr3)-1::-1]
'noitacilppA'
Or
substr3[::-1]
'noitacilppA'
```

We want to do reverse, but we got empty string because the default movement is forward direction. Therefore we need to change the direction by using 3rd parameter. If we put '-' in the 3rd parameter then direction will be reversed.

```
substr3
'Application'
substr3[:5]
'Appli'
substr3[5:8]
'cat'
substr3[::2]
'Apiain'
substr3[::-2]
'niaipA'
```

Traversing a String

We can access each character of a string or traverse a string using for loop and while loop.

String traversal using for loop:

```
for c in substr3:
    print(c)
```

A
p

p
l
i
c
a
t
i
o
n

String traversal using while loop

For example:

```
index = 0
while index < len(substr3):
    print(substr3[index])
    index += 1
```

A
p
p
l
i
c
a
t
i
o
n

String methods and built in functions

Python has several built in functions that allow us to work with strings.

Method	Description	Example
<code>len()</code>	Returns the length of the given string.	<code>substr3</code> <code>'Application'</code> <code>len(substr3)</code> <code>11</code>
<code>title()</code>	Returns the string with the first letter of every word in the string in uppercase and rest in lowercase.	<code>str2 = 'hi shiv!, how are you?'</code> <code>tstr2 = str2.title()</code> <code>tstr2</code> <code>'Hi Shiv!, How Are You?'</code>
<code>lower()</code>	Returns the string with all uppercase letters converted to lowercase, and lowercase will remain at lowercase.	<code>tstr2</code> <code>'Hi Shiv!, How Are You?'</code> <code>ltstr2 = tstr2.lower()</code> <code>ltstr2</code> <code>'hi shiv!, how are you?'</code>
<code>upper()</code>	He returns the string with all lowercase letters converted to uppercase. And upper case will	<code>tstr2</code> <code>'Hi Shiv!, How Are You?'</code> <code>uptstr2 = tstr2.upper()</code>

	remain at upper case.	uptstr2 'HI SHIV!, HOW ARE YOU?'
count(str, start, end)	It returns number of times substring STR occurs in the given string. If we do not give the start index and end index then searching starts from index 0 and ends at length of the string.	tstr2 'Hi Shiv!, How Are You?' count0 = tstr2.count('o') count0 2 count0 = tstr2.count('i') count0 2 count0 = tstr2.count('b') count0 0 count0 = tstr2.count('o', -5, -1) count0 1